

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторной работы №3
По дисциплине: «Программирование микроконтроллеров»
На тему: «События и прерывания»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверила
к.т.н. доцент

_____ Верובה Н. М.

«_____» _____ 2026г.

Содержание

1	Цель работы	3
2	Постановка задачи	3
3	Выполнение работы	3
3.1	Функция задержки	3
3.2	Обработчик прерывания WAKEUP	3
3.3	Обработчик прерывания USER	4
3.4	Настройка тактирования и GPIO	4
3.5	Настройка системы прерываний EXTI	5
3.6	Настройка NVIC и приоритетов	6
3.7	Основной цикл программы	6
4	Вывод	6
	Приложение. Полный текст программы	7

1 Цель работы

Ознакомление с основными методами обработки событий и прерываний в микроконтроллерах.

2 Постановка задачи

Разработать программу для микроконтроллера STM32F200, обеспечивающую мигание светодиода PG7, а также обработку нажатий кнопок WAKEUP и USER с использованием прерываний. При нажатии кнопки WAKEUP должен включаться светодиод PG6, а при нажатии кнопки USER — светодиод PG8. Обработка прерываний должна выполняться с разными приоритетами.

3 Выполнение работы

В среде Keil μ Vision5 был создан проект, в котором реализована программа обработки внешних прерываний и управления светодиодами.

3.1 Функция задержки

Для создания временных задержек используется функция `delay()`. Она реализована в виде пустого цикла; переменная цикла объявлена с квалификатором `volatile`, чтобы компилятор не оптимизировал цикл.

Листинг 1: Функция задержки

```
1 void delay(unsigned long time)
2 {
3     volatile unsigned long i;
4     for (i = 0; i < time; i++) {}
5 }
```

3.2 Обработчик прерывания WAKEUP

Обработчик `EXTI0_IRQHandler` вызывается при нажатии кнопки WAKEUP, подключенной к выводу PA0. В начале работы обработчика проверяется флаг прерывания в регистре `EXTI->PR`. После подтверждения события включается светодиод PG6, выполняется задержка, светодиод выключается, а флаг прерывания сбрасывается.

Листинг 2: Обработчик прерывания WAKEUP

```
1 void EXTI0_IRQHandler(void)
2 {
3     if (EXTI->PR & EXTI_PR_PR0) {
4         GPIOG->ODR |= (1 << 6); /* Включить светодиод PG6 */
5         delay(3000000);
6         GPIOG->ODR &= ~(1 << 6); /* Выключить светодиод PG6 */
7         EXTI->PR = EXTI_PR_PR0; /* Сброс флага прерывания */
8     }
9 }
```

3.3 Обработчик прерывания USER

Обработчик EXTI15_10_IRQHandler срабатывает при нажатии кнопки USER, подключенной к выводу PG15. Так как данный обработчик обслуживает несколько линий EXTI, дополнительно проверяется, что событие произошло именно на линии 15. Дальнейшая логика аналогична обработчику WAKEUP: включение светодиода PG8, задержка, выключение и сброс флага прерывания.

Листинг 3: Обработчик прерывания USER

```
1 void EXTI15_10_IRQHandler(void)
2 {
3     if (EXTI->PR & EXTI_PR_PR15) {
4         GPIOG->ODR |= (1 << 8); /* Включить светодиод PG8 */
5         delay(3000000);
6         GPIOG->ODR &= ~(1 << 8); /* Выключить светодиод PG8 */
7         EXTI->PR = EXTI_PR_PR15; /* Сброс флага прерывания */
8     }
9 }
```

3.4 Настройка тактирования и GPIO

На первом этапе работы программы включается тактирование портов GPIOA и GPIOG. Светодиоды PG6, PG7 и PG8 настраиваются как выходы, а кнопки PA0 и PG15 — как входы.

Листинг 4: Настройка тактирования и GPIO

```
1  /* Включение тактирования портов GPIOA и GPIOG */
2  RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOGEN;
3
4  /* PG6, PG7, PG8 -> выход */
5  GPIOG->MODER &= ~(3 << (6 * 2));
6  GPIOG->MODER |= (1 << (6 * 2));
7
8  GPIOG->MODER &= ~(3 << (7 * 2));
9  GPIOG->MODER |= (1 << (7 * 2));
10
11 GPIOG->MODER &= ~(3 << (8 * 2));
12 GPIOG->MODER |= (1 << (8 * 2));
13
14 /* PA0 -> вход (кнопка WAKEUP) */
15 GPIOA->MODER &= ~(3 << (0 * 2));
16
17 /* PG15 -> вход (кнопка USER) */
18 GPIOG->MODER &= ~(3 << (15 * 2));
```

3.5 Настройка системы прерываний EXTI

Для работы внешних прерываний включается модуль SYSCFG и настраиваются линии EXTI. Линия EXTI0 связывается с выводом PA0, а линия EXTI15 — с выводом PG15. После этого разрешается генерация прерываний и задается срабатывание по переднему и заднему фронтам сигнала через регистры RTSR и FTSR.

Листинг 5: Настройка EXTI

```
1  /* Включить тактирование модуля SYSCFG */
2  RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
3
4  /* Привязать EXTI0 к PA0 и EXTI15 к PG15 */
5  SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTIO_PA;
6  SYSCFG->EXTICR[3] |= SYSCFG_EXTICR4_EXTI15_PG;
7
8  /* Разрешить прерывания по линиям 0 и 15 */
9  EXTI->IMR |= (1 << 0) | (1 << 15);
10
11 /* Срабатывание по переднему фронту */
12 EXTI->RTSR |= (1 << 0) | (1 << 15);
13
14 /* Срабатывание по заднему фронту */
15 EXTI->FTSR |= (1 << 0) | (1 << 15);
```

3.6 Настройка NVIC и приоритетов

Контроллер NVIC используется для управления прерываниями и задания их приоритетов. Прерыванию от кнопки USER назначается более высокий приоритет, чем прерыванию от кнопки WAKEUP. Это означает, что при одновременном возникновении событий сначала будет обработано прерывание USER.

Листинг 6: Настройка NVIC

```
1 NVIC_SetPriority(EXTI15_10_IRQn, 0); /* USER: высокий приоритет */
2 NVIC_SetPriority(EXTIO_IRQn, 1); /* WAKEUP: низкий приоритет */
3
4 NVIC_EnableIRQ(EXTIO_IRQn);
5 NVIC_EnableIRQ(EXTI15_10_IRQn);
```

3.7 Основной цикл программы

В основном цикле программы реализовано мигание светодиода PG7. Цикл выполняется бесконечно, однако может быть прерван в любой момент при возникновении внешнего прерывания от одной из кнопок.

Листинг 7: Основной цикл программы

```
1 while (1) {
2     GPIOG->ODR |= (1 << 7); /* Включить PG7 */
3     delay(2000000);
4     GPIOG->ODR &= ~(1 << 7); /* Выключить PG7 */
5     delay(2000000);
6 }
```

4 Вывод

В ходе выполнения лабораторной работы была разработана программа для микроконтроллера STM32F200, реализующая обработку внешних прерываний. Были изучены принципы работы системы EXTI и контроллера NVIC, а также механизм приоритетов прерываний. Практически продемонстрировано, что прерывания с более высоким приоритетом обрабатываются раньше. Полученные навыки позволяют использовать прерывания для эффективной обработки внешних событий в микроконтроллерных системах.

Приложение. Полный текст программы

```
1  #include "stm32f2xx.h"
2
3  void delay(unsigned long time)
4  {
5      volatile unsigned long i;
6      for (i = 0; i < time; i++) {}
7  }
8
9  void EXTI0_IRQHandler(void)
10 {
11     if (EXTI->PR & EXTI_PR_PRO) {
12         GPIOG->ODR |= (1 << 6);
13         delay(3000000);
14         GPIOG->ODR &= ~(1 << 6);
15         EXTI->PR = EXTI_PR_PRO;
16     }
17 }
18
19 void EXTI15_10_IRQHandler(void)
20 {
21     if (EXTI->PR & EXTI_PR_PR15) {
22         GPIOG->ODR |= (1 << 8);
23         delay(3000000);
24         GPIOG->ODR &= ~(1 << 8);
25         EXTI->PR = EXTI_PR_PR15;
26     }
27 }
28
29 int main(void)
30 {
31     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOGEN;
32     RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
33
34     GPIOG->MODER &= ~(3 << (6 * 2));
35     GPIOG->MODER |= (1 << (6 * 2));
36     GPIOG->MODER &= ~(3 << (7 * 2));
37     GPIOG->MODER |= (1 << (7 * 2));
38     GPIOG->MODER &= ~(3 << (8 * 2));
39     GPIOG->MODER |= (1 << (8 * 2));
40     GPIOA->MODER &= ~(3 << (0 * 2));
41     GPIOG->MODER &= ~(3 << (15 * 2));
42
43     SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTIO_PA;
44     SYSCFG->EXTICR[3] |= SYSCFG_EXTICR4_EXTI15_PG;
45
46     EXTI->IMR |= (1 << 0) | (1 << 15);
47     EXTI->RTSR |= (1 << 0) | (1 << 15);
48     EXTI->FTSR |= (1 << 0) | (1 << 15);
49
50     NVIC_SetPriority(EXTI15_10_IRQn, 0);
```

```

51     NVIC_SetPriority(EXTIO_IRQn, 1);
52     NVIC_EnableIRQ(EXTIO_IRQn);
53     NVIC_EnableIRQ(EXTI15_10_IRQn);
54
55     while (1) {
56         GPIOG->ODR |= (1 << 7);
57         delay(2000000);
58         GPIOG->ODR &= ~(1 << 7);
59         delay(2000000);
60     }
61 }

```